Lab Exercise

Name-Himanshu Khatik

Rollno-19

Subjects-DSA & Python

DSA Programs

Q1. Write a program to implement Singly linked list with required member function(constructor, insertfirst, insertpos, insertlast, deletefirst, deletelast, deletepos, Display)

→

```
class Node:
    def __init__(self,data):
        self.data=data
        self.next=None


class SLL:
    def __init__(self):
        self.head=None


    def insert_beginning(self,data):
        n=Node(data)
        n.next=self.head
        self.head=n


    def insert_end(self,data):
        n=Node(data)
        temp=self.head
        while temp.next is not None:
            temp=temp.next
        temp.next=n
```

```
def insert_pos(self,pos,data):
    n=Node(data)
    temp=self.head
    for i in range(pos-1):
        temp=temp.next
    n.data=data
    n.next=temp.next
    temp.next=n

def delete_beginning(self):
    temp=self.head
    self.head=temp.next
    temp.next=None

def delete_end(self):
    prev=self.head
    temp=self.head.next
    while temp.next is not None:
        temp=temp.next
        prev=prev.next
    prev.next=None

def delete_pos(self,pos):
     prev=self.head
     temp=self.head.next
     for i in range(pos-1):
         temp=temp.next
         prev=prev.next
```

```
        prev.next=temp.next


    def display(self):
        if self.head is None:
            print("Linkedlist is empty")
        else:
            temp=self.head
            while temp:
                print(temp.data,end="->")
                temp=temp.next
sll=SLL()
n1=Node(10)
sll.head=n1
n2=Node(20)
n1.next=n2
n3=Node(30)
n2.next=n3
sll.insert_beginning(5)
sll.insert_end(40)
sll.insert_pos(3,25)
# sll.delete_beginning()
# sll.delete_end()
# sll.delete_pos(2)
sll.display()
#Output:
```

```
5->10->20->25->30->40->
```

```
]:
```

Q2. Write a program to implement Doubly linked list with required data members and member function(constructor, insertfirst, insertpos, insertlast, deletefirst, deletelast, deletepos, Display)

→

```
class Node:
    def __init__(self,data):
        self.data=data
        self.prev=None
        self.next=None

class DLL:
    def __init__(self):
        self.head=None

    def display(self):
        if self.head is None:
            print("Linkedlist is empty")
        else:
            temp=self.head
            while temp:
                print(temp.data,end="->")
                temp=temp.next

    def insert_beginning(self,data):
        n=Node(data)
        temp=self.head
        temp.prev=n
        n.next=temp
        self.head=n
```

```
def insert_end(self,data):
    n=Node(data)
    temp=self.head
    while temp.next is not None:
        temp=temp.next
    temp.next=n
    n.prev=temp

def insert_pos(self,pos,data):
    n=Node(data)
    temp=self.head
    for i in range(pos-1):
        temp=temp.next
    n.prev=temp
    n.next=temp.next
    temp.next.prev=n
    temp.next=n

def delete_beginning(self):
    temp=self.head
    self.head=temp.next
    temp.next=None
    self.head.prev=None

def delete_end(self):
    temp=self.head.next
    before=self.head
    while temp.next is not None:
        temp=temp.next
```

```
            before=before.next
        before.next=None
        temp.prev=None

    def delete_pos(self,pos):
        temp=self.head.next
        before=self.head
        for i in range(pos-1):
            temp=temp.next
            before=before.next
        before.next=temp.next
        temp.next.prev=before
        temp.next=None
        temp.prev=None


dll=DLL()
n1=Node(10)
dll.head=n1
n2=Node(20)
n2.prev=n1
n1.next=n2
n3=Node(30)
n3.prev=n2
n2.next=n3
n4=Node(40)
n4.prev=n3
n3.next=n4
dll.insert_beginning(5)
```

```
dll.insert_end(50)
dll.insert_pos(3,25)
dll.delete_beginning()
dll.delete_end()
dll.delete_pos(2)
dll.display()
```

#Output:

```
10->20->30->40->
```

Q3. Write a program to implement Circular linked list with required data members and member function(constructor, insertfirst, insertpos, insertlast, deletefirst, deletelast, deletepos, Display)

→

```
class Node:
    def __init__(self,data):
        self.data=data
        self.next=None

class CLL:
    def __init__(self):
        self.head=None
        self.tail=None

    def display(self):
        if self.head is None:
            print("Circular linked list is empty")
        else:
```

```
        temp=self.head
        print(temp.data,"-->",end=" ")
        while temp.next!=self.head:
            temp=temp.next
            print(temp.data,"-->",end=" ")
        print(temp.next.data)

    def add_begin(self,data):
        new=Node(data)
        if self.head is None:
            self.head=new
            self.tail=new
            self.tail.next=self.head
        else:
            new.next=self.head
            self.tail.next=new
            self.head=new

    def add_end(self,data):
        new=Node(data)
        if self.head is None:
            self.head=new
            self.tail=new
            self.tail.next=self.head
        else:
            self.tail.next=new
            self.tail=new
            self.tail.next=self.head
```

```
def add_position(self,pos,data):
    new=Node(data)
    if self.head is None:
        self.head=new
        self.tail=new
        self.tail.next=self.head
    else:
        if pos==0:
            cll.add_begin(data)
        else:
            temp=self.head
            for i in range(pos-1):
                temp=temp.next
            new.next=temp.next
            temp.next=new

def delete_begin(self):
    if self.head is None:
        print("No node in CLL")
    else:
        if self.head==self.tail:
            self.head=None
        else:
            temp=self.head
            self.head=temp.next
            temp=None
            self.tail.next=self.head

def delete_end(self):
```

```
    if self.head is None:
        print("No node in CLL")
    else:
        if self.head==self.tail:
            self.head=None
        else:
            temp=self.head
            while temp.next!=self.tail:
                temp=temp.next
            self.tail=None
            self.tail=temp
            temp.next=self.head

def delete_pos(self,pos):
    if self.head is None:
        print("No node in CLL")
    elif pos==0:
        cll.add_begin()
    else:
        temp1=self.head
        temp2=self.head.next
        for i in range(pos-1):
            temp1=temp1.next
            temp2=temp2.next
            temp1.next=temp2.next
            if temp2==self.tail:
                self.tail=temp1
            temp2=None
```

```
cll=CLL()
cll.add_begin(10)
cll.add_begin(20)
cll.add_begin(30)
cll.add_end(5)
cll.add_position(2,15)
cll.add_position(0,40)
cll.delete_begin()
cll.display()
cll.delete_end()
cll.display()
cll.delete_pos(2)
cll.display()
```

#Output:

```
30 --> 20 --> 15 --> 10 --> 5 --> 30
30 --> 20 --> 15 --> 10 --> 30
30 --> 20 --> 10 --> 30
```

Q4. Write a program to reverse singly linked list, Doubly linked list.

→

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SLL:
    def __init__(self):
        self.head = None
```

```
    def insert_end(self, data):
        n = Node(data)
        temp = self.head
        if temp is None:
            self.head = n
            return
        while temp.next is not None:
            temp = temp.next
        temp.next = n

    def display(self):
        if self.head is None:
            print("Linkedlist is empty")
        else:
            temp = self.head
            while temp:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

def reverse_singly(sll):
    prev = None
    current = sll.head
    while current:
        nxt = current.next
        current.next = prev
        prev = current
        current = nxt
    sll.head = prev
```

```
class Node1:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DLL:
    def __init__(self):
        self.head = None

    def insert_end(self, data):
        n = Node1(data)
        temp = self.head
        if temp is None:
            self.head = n
            return
        while temp.next is not None:
            temp = temp.next
        temp.next = n
        n.prev = temp

    def display(self):
        if self.head is None:
            print("Circular linked list is empty")
        else:
            temp = self.head
            while temp:
                print(temp.data, end=" -> ")
```

```
            temp = temp.next
        print("None")

def reverse_doubly(dll):
    current = dll.head
    temp = None
    while current:
        temp = current.prev
        current.prev = current.next
        current.next = temp
        current = current.prev
    if temp:
        dll.head = temp.prev

print("Singly")
sll = SLL()
sll.insert_end(10)
sll.insert_end(20)
sll.insert_end(30)
print("Original")
sll.display()
reverse_singly(sll)
print("Reversed")
sll.display()

print("\nDoubly")
dll = DLL()
dll.insert_end(11)
dll.insert_end(22)
```

```
dll.insert_end(33)
print("Original")
dll.display()
reverse_doubly(dll)
print("Reversed")
dll.display()
```

#Output:

```
Singly
Original
10 -> 20 -> 30 -> None
Reversed
30 -> 20 -> 10 -> None

Doubly
Original
11 -> 22 -> 33 -> None
Reversed
33 -> 22 -> 11 -> None
```

Q5. Write a program to implement stack using Array with PUSH,POP operations.

→

```
class StackArray:
    def __init__(self, size):
        self.size = size
        self.stack = []

    def push(self, data):
        if len(self.stack) == self.size:
            print("Stack Overflow")
        else:
            self.stack.append(data)

    def pop(self):
```

```
        if len(self.stack) == 0:
            print("Stack Underflow")
        else:
            return self.stack.pop()

    def display(self):
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            for i in range(len(self.stack)-1, -1, -1):
                print(self.stack[i], end=" -> ")
            print("None")



print("Stack Using Array")

s = StackArray(5)
s.push(10)
s.push(20)
s.push(30)

print("After Push")
s.display()

print("Popped:", s.pop())

print("After Pop")
s.display()
```

#Output:

```
Stack Using Array
After Push
30 -> 20 -> 10 -> None
Popped: 30
After Pop
20 -> 10 -> None
```

Q6. Write a program to implement stack using Linked list with PUSH,POP,PEEK and Display operations.

→

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class StackLL:
    def __init__(self):
        self.top = None

    def push(self, data):
        n = Node(data)
        n.next = self.top
        self.top = n

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            val = self.top.data
            self.top = self.top.next
            return val
```

```
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            return self.top.data

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top
            while temp:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

print("Stack Using Linked List")
s = StackLL()
s.push(10)
s.push(20)
s.push(30)
print("After Push")
s.display()
print("Peek:", s.peek())
print("Popped:", s.pop())
print("After Pop")
s.display()
```

#Output:

```
Stack Using Linked List
After Push
30 -> 20 -> 10 -> None
Peek: 30
Popped: 30
After Pop
20 -> 10 -> None
```

Q7. Write a application of stack to check for balanced parentheses.

Write a program to reverse a string using stack.

→(i)

```
def is_balanced(expr):
    stack = []
    for ch in expr:
        if ch in "({[":
            stack.append(ch)
        elif ch in ")}]":
            if not stack:
                return False
            top = stack.pop()
            if (ch == ')' and top != '(') or \
              (ch == ']' and top != '[') or \
              (ch == '}' and top != '{'):
                return False
    return len(stack) == 0

expr = "{(a+b) * (c+d)}"
print("Expression:", expr)

if is_balanced(expr):
    print("Balanced")
```

```
else:
    print("Not Balanced")
```

#Output:

```
Expression: {(a+b) * (c+d)}
Balanced
```

→(ii)

```
def reverse_string(s):
    stack = []
    for ch in s:
        stack.append(ch)
    rev = ""
    while stack:
        rev += stack.pop()
    return rev

s = "hello"
print("Original:", s)
print("Reversed:", reverse_string(s))
```

#Output:

```
Original: hello
Reversed: olleh
```

Q8. Write a program to implement

1.Linear Queue using list 2.Circular Queue using list

→(1) Linear Queue

```
class LinearQueue:
```

```
    def __init__(self):
        self.q = []

    def enqueue(self, data):
        self.q.append(data)

    def dequeue(self):
        if not self.q:
            print("Queue is empty")
        else:
            return self.q.pop(0)

    def display(self):
        print(self.q)

q = LinearQueue()
q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
q.display()
q.dequeue()
q.display()
```

#Output:

```
[10, 20, 30]
[20, 30]
```

→(2) Circular Queue

```
class CircularQueue:
    def __init__(self, size):
```

```
    self.size = size
    self.q = [None] * size
    self.front = -1
    self.rear = -1

def enqueue(self, data):
    if (self.rear + 1) % self.size == self.front:
        print("Queue is full")
    else:
        if self.front == -1:
            self.front = 0
        self.rear = (self.rear + 1) % self.size
        self.q[self.rear] = data

def dequeue(self):
    if self.front == -1:
        print("Queue is empty")
    else:
        val = self.q[self.front]
        if self.front == self.rear:
            self.front = self.rear = -1
        else:
            self.front = (self.front + 1) % self.size
        return val

def display(self):
    if self.front == -1:
        print("[]")
    else:
```

```
        i = self.front
        while True:
            print(self.q[i], end=" ")
            if i == self.rear:
                break
            i = (i + 1) % self.size
        print()

cq = CircularQueue(5)
cq.enqueue(10)
cq.enqueue(20)
cq.enqueue(30)
cq.display()
cq.dequeue()
cq.display()
```

#Output:

```
10 20 30
20 30
```

Q9. Write a program to Reverse stack using queue.

→

```
class Stack:
    def __init__(self):
        self.s = []
    def push(self, data):
        self.s.append(data)
    def pop(self):
        if not self.s:
```

```
            return None
        return self.s.pop()
    def display(self):
        print(self.s)

class Queue:
    def __init__(self):
        self.q = []
    def enqueue(self, data):
        self.q.append(data)
    def dequeue(self):
        if not self.q:
            return None
        return self.q.pop(0)

def reverse_stack(stack):
    q = Queue()
    while stack.s:
        q.enqueue(stack.pop())
    while q.q:
        stack.push(q.dequeue())

s = Stack()
s.push(10)
s.push(20)
s.push(30)

print("Original Stack:")
s.display()
```

```
reverse_stack(s)
print("Reversed Stack:")
s.display()
```

#output:

```
Original Stack:
[10, 20, 30]
Reversed Stack:
[30, 20, 10]
```

Q10. Write a program to implement binary search tree with its operations.

→

```
class node:
    def __init__(self,val):
        self.lchild=None
        self.data=val
        self.rchild=None
class btree:
    def __init__(self):
        self.root=None
    def insert(self,val):
        temp=node(val)
        trv=self.root
        if self.root is None:
            self.root=temp
            return
        else:
            while True:
                if(val > trv.data):
                    if trv.rchild is None:
```

```
                trv.rchild=temp
                return
            else:
                trv=trv.rchild
        else:
            if trv.lchild is None:
                trv.lchild=temp
                return
            else:
                trv=trv.lchild

def search(self,key):
    trv=self.root
    while True:
        if trv is None:
            return False
        if trv.data==key:
            return True
        if trv.data<key:
            trv=trv.rchild
        else:
            trv=trv.lchild

def searchR(self, root, key):
    if root is None:
        return False
    elif key < root.data:
        return self.search(root.lchild,key)
    elif key > root.data:
```

```
        return self.search(root.rchild,key)
    else:
        if root.data==key:
            return True
        else:
            return False

def inorder(self,rt):
    if rt is not None:
        self.inorder(rt.lchild)
        print(rt.data,end=" ")
        self.inorder(rt.rchild)
def preorder(self,rt):
    if rt is not None:
        print(rt.data,end=" ")
        self.preorder(rt.lchild)
        self.preorder(rt.rchild)
def postorder(self,rt):
    if rt is not None:
        self.postorder(rt.lchild)
        self.postorder(rt.rchild)
        print(rt.data,end=" ")
def inorder_traversal(self):
    stack = []
    current = self.root
    result = []
    while current or stack:
        while current:
            stack.append(current)
```

```
            current = current.lchild
        current = stack.pop()
        result.append(current.data)
        current = current.rchild
    return result
def preorder_traversal(self):
    if not self.root:
        return []
    stack, result = [self.root], []
    while stack:
        node = stack.pop()
        result.append(node.data)

    # Push right child first so that left is processed first
        if node.rchild:
            stack.append(node.rchild)
        if node.lchild:
            stack.append(node.lchild)

    return result
def post_order_traversal(self):
    if self.root is None:
        return []

    stack1 = []
    stack2 = []
    stack1.append(self.root)

    while stack1:
```

```
        node = stack1.pop()
        stack2.append(node)

        if node.lchild:
            stack1.append(node.lchild)
        if node.rchild:
            stack1.append(node.rchild)

    result = []
    while stack2:
        node = stack2.pop()
        result.append(node.data)

    return result

bst=btree()
bst.insert(10)
bst.insert(5)
bst.insert(20)
bst.insert(50)
bst.insert(30)
bst.insert(40)
bst.insert(35)
print("Inorder")
#print(bst.inorder_traversal())
bst.inorder(bst.root)
#bst.displayInorder()
print("\nPreorder")
#print(bst.preorder_traversal())
```

```
bst.preorder(bst.root)
print("\nPostorder")
#print(bst.post_order_traversal())
bst.postorder(bst.root)
print("\nSearch for 10 in the BST:", bst.search(10))
```

#Output:

```
Inorder
5 10 20 30 35 40 50
Preorder
10 5 20 50 30 40 35
Postorder
5 35 40 30 50 20 10
Search for 10 in the BST: True
```

Q11. Write a program to implement AVL Tree.

→

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
        self.height = 1

class AVLTree:
    def __init__(self):
        self.root = None

    def height(self, node):
        if not node:
            return 0
        return node.height
```

```
def balance(self, node):
    if not node:
        return 0
    return self.height(node.left) - self.height(node.right)

def insert(self, root, value):
    if not root:
        return Node(value)
    elif value < root.value:
        root.left = self.insert(root.left, value)
    else:
        root.right = self.insert(root.right, value)

    root.height = 1 + max(self.height(root.left), self.height(root.right))
    balance = self.balance(root)

    # Left rotation
    if balance > 1 and value < root.left.value:
        return self.right_rotate(root)

    # Right rotation
    if balance < -1 and value > root.right.value:
        return self.left_rotate(root)

    # Left-Right rotation
    if balance > 1 and value > root.left.value:
        root.left = self.left_rotate(root.left)
        return self.right_rotate(root)
```

```
    # Right-Left rotation
    if balance < -1 and value < root.right.value:
        root.right = self.right_rotate(root.right)
        return self.left_rotate(root)

    return root

def delete(self, root, value):
    if not root:
        return root

    if value < root.value:
        root.left = self.delete(root.left, value)
    elif value > root.value:
        root.right = self.delete(root.right, value)
    else:
        if not root.left:
            temp = root.right
            root = None
            return temp
        elif not root.right:
            temp = root.left
            root = None
            return temp

        temp = self.min_value_node(root.right)
        root.value = temp.value
        root.right = self.delete(root.right, temp.value)
```

```
    if not root:
        return root

    root.height = 1 + max(self.height(root.left), self.height(root.right))
    balance = self.balance(root)

    # Left rotation
    if balance > 1 and self.balance(root.left) >= 0:
        return self.right_rotate(root)

    # Right rotation
    if balance < -1 and self.balance(root.right) <= 0:
        return self.left_rotate(root)

    # Left-Right rotation
    if balance > 1 and self.balance(root.left) < 0:
        root.left = self.left_rotate(root.left)
        return self.right_rotate(root)

    # Right-Left rotation
    if balance < -1 and self.balance(root.right) > 0:
        root.right = self.right_rotate(root.right)
        return self.left_rotate(root)

    return root

def left_rotate(self, z):
    y = z.right
```

```
    T2 = y.left

    y.left = z
    z.right = T2

    z.height = 1 + max(self.height(z.left), self.height(z.right))
    y.height = 1 + max(self.height(y.left), self.height(y.right))

    return y

def right_rotate(self, z):
    y = z.left
    T3 = y.right

    y.right = z
    z.left = T3

    z.height = 1 + max(self.height(z.left), self.height(z.right))
    y.height = 1 + max(self.height(y.left), self.height(y.right))

    return y

def min_value_node(self, root):
    current = root
    while current.left:
        current = current.left
    return current

def search(self, root, value):
```

```
        if not root or root.value == value:
            return root
        if root.value < value:
            return self.search(root.right, value)
        return self.search(root.left, value)

    def insert_value(self, value):
        self.root = self.insert(self.root, value)

    def delete_value(self, value):
        self.root = self.delete(self.root, value)

    def search_value(self, value):
        return self.search(self.root, value)

    # In-order traversal to print the tree
    def inorder_traversal(self,root):
        if root:
            self.inorder_traversal(root.left)
            print(root.value),
            self.inorder_traversal(root.right)


tree = AVLTree()
tree.insert_value(10)
tree.insert_value(20)
tree.insert_value(30)
tree.insert_value(40)
tree.insert_value(50)
```

```
print("Tree after insertion:")
tree.inorder_traversal(tree.root)
print()
##tree.delete_value(20)
##print("Tree after deletion of 20:")
##tree.inorder_traversal(tree.root)
##print()
result = tree.search_value(30)
if result:
    print("Node found")
else:
    print("Node not found")
```

#Output:

```
Tree after insertion:
10
20
30
40
50

Node found
```

Q12. Write a program to implement Graph

1.Adjacency list

2.Adjacency matrix

→(1) Adjacency list

```
class GraphList:
    def __init__(self):
        self.graph = {}


    def add_edge(self, u, v):
```

```
        if u not in self.graph:
            self.graph[u] = []
        self.graph[u].append(v)

    def display(self):
        for node in self.graph:
            print(node, "->", self.graph[node])

g = GraphList()
g.add_edge(1, 2)
g.add_edge(1, 3)
g.add_edge(2, 4)
g.add_edge(3, 4)
g.display()
```

#Output:

```
1 -> [2, 3]
2 -> [4]
3 -> [4]
```

→(2) Adjacency matrix

```
class GraphMatrix:
    def __init__(self, n):
        self.n = n
        self.mat = [[0]*n for _ in range(n)]

    def add_edge(self, u, v):
        self.mat[u][v] = 1

    def display(self):
        for row in self.mat:
```

```
        print(row)


g = GraphMatrix(4)
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 3)
g.add_edge(2, 3)
g.display()


#Output:
```

```
[0, 1, 1, 0]
[0, 0, 0, 1]
[0, 0, 0, 1]
[0, 0, 0, 0]
```

Q13. Write a program to find the element in an array using

1. Binary search
2. Linear search

→(1) Binary search

```
def binary_search(arr, x):
    low = 0
    high = len(arr) - 1


    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == x:
            return mid
        elif arr[mid] < x:
            low = mid + 1
        else:
            high = mid - 1
```

```
    return -1

arr = [10, 20, 30, 40, 50]
x = 25

result = binary_search(arr, x)
if result == -1:
    print("Element not found")
else:
    print("Element found at index:", result)

#Output:
```

```
Element not found
```

→(2) Linear search

```
def linear_search(arr, x):
    for i in range(len(arr)):
        if arr[i] == x:
            return i
    return -1

arr = [10, 20, 30, 40, 50]
x = 90

result = linear_search(arr, x)
if result == -1:
    print("Element not found")
else:
    print("Element found at index:", result)
```

#Output:

```
Element not found
```

Q14. Write a program to implement sorting using Divide and Conquer strategy

1. Quick sort 2. Merge sort

→(1) Quick sort

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[0]
    left = [x for x in arr[1:] if x < pivot]
    right = [x for x in arr[1:] if x >= pivot]
    return quick_sort(left) + [pivot] + quick_sort(right)

arr = [40, 10, 80, 20, 60]
print("Original:", arr)
print("Sorted:", quick_sort(arr))
```

#Output:

```
Original: [40, 10, 80, 20, 60]
Sorted: [10, 20, 40, 60, 80]
```

→(2) Merge sort

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
```

```
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    i = j = 0
    result = []

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

arr = [40, 10, 80, 20, 60]
print("Original:", arr)
print("Sorted:", merge_sort(arr))

#Output:
```

```
Original: [40, 10, 80, 20, 60]
Sorted: [10, 20, 40, 60, 80]
```

Python programs

Q1. Write Python Program to count the Total Number of Vowels, Consonants and Blanks in a String.

Create Stack using list and implement necessary operation.

→(1)

```
string = input("Enter a string: ")


vowels = "aeiouAEIOU"

v_count = c_count = b_count = 0


for ch in string:

    if ch in vowels:

        v_count += 1

    elif ch == " ":

        b_count += 1

    elif ch.isalpha():

        c_count += 1


print("Vowels:", v_count)

print("Consonants:", c_count)

print("Blanks:", b_count)


#Output:
```

```
Enter a string:  Himanshu 123
Vowels: 3
Consonants: 5
Blanks: 1
```

→(2)

```
stack = []
```

```
def push():
    item = input("Enter element to push: ")
    stack.append(item)
    print(stack)

def pop():
    if not stack:
        print("Stack is empty!")
    else:
        stack.pop()
        print(stack)

while True:
    ch = int(input("\n1 Push\n2 Pop\n3 Exit\nEnter choice: "))
    if ch == 1: push()
    elif ch == 2: pop()
    else: break

#Output:
```

```
1 Push
2 Pop
3 Exit
Enter choice:  1
Enter element to push:  11
['11']

1 Push
2 Pop
3 Exit
Enter choice:  1
Enter element to push:  22
['11', '22']

1 Push
2 Pop
3 Exit
Enter choice:  2
['11']

1 Push
2 Pop
3 Exit
Enter choice:  ↑↓ for history.
```

Q2. Write a program to print following pattern.

A	4444	1
B C	333	0 1
C D E	22	1 0 1
D E F G	1	0 1 0 1

→(1)

```
for i in range(4):
    for j in range(i + 1):
        print(chr(65 + i + j), end=" ")
    print()
```

#Output:

```
A
B C
C D E
D E F G
```

→(2)

```
num = 4
for i in range(4):
    for j in range(4 - i):
        print(num - i, end="")
    print()
```

#Output:

```
4444
333
22
1
```

→(3)

```
for i in range(4):
    num = 1
    for j in range(i + 1):
        print(num, end=" ")
        num = 1 - num
    print()
```

#Output:

```
1
1 0
1 0 1
1 0 1 0
```

Q3. Write a program which generates account number using random function.

→

```
import random

name = input("Enter your name: ")

acc_number = random.randint(100000, 999999)

print("\nHello", name + "!")

print("Your Account Number is:", acc_number)
```

#Output:

```
Enter your name:  Himanshu

Hello Himanshu!
Your Account Number is: 160348
```

Q4. Write a program to conduct a linear search for a given key number in the list and report success or failure.

→

```
lst = list(map(int, input("Enter list elements separated by space: ").split()))

key = int(input("Enter the number to search: "))


flag = False


for i in range(len(lst)):

    if lst[i] == key:

        print("Success! Number found at position:", i + 1)

        flag = True

        break


if not flag:

    print("Failure! Number not found in list.")
```

#Output:

```
Enter list elements separated by space:  10 20 30 40 50
Enter the number to search:  30
Success! Number found at position: 3
```

Q5. Write a python program to add two matrices.

→

```
r = int(input("Enter number of rows: "))

c = int(input("Enter number of columns: "))


print("Enter elements of Matrix A:")

A = [list(map(int, input().split())) for _ in range(r)]


print("Enter elements of Matrix B:")

B = [list(map(int, input().split())) for _ in range(r)]


# Resultant Matrix

C = [[A[i][j] + B[i][j] for j in range(c)] for i in range(r)]


print("\nResultant Matrix (A + B) :")

for row in C:

    print(*row)
```

#Output:

```
Enter number of rows:  2
Enter number of columns:  2
Enter elements of Matrix A:
 1 2
 3 4
Enter elements of Matrix B:
 1 2
 3 4

Resultant Matrix (A + B) :
2 4
6 8
```

Q6. Write a program to create point class with x,y,z coordinate and methods increment point, decrement point, add points, less than, greater than, equal to, check in which quadrant it lies, check whether the point is collinear and print point.

→

```
class Point:
    def __init__(self, x, y, z):
        self.x = x
        self.y = y
        self.z = z

    def inc(self):
        self.x += 1
        self.y += 1
        self.z += 1

    def dec(self):
        self.x -= 1
        self.y -= 1
        self.z -= 1

    def show(self):
        print(f"Point: ({self.x}, {self.y}, {self.z})")

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y, self.z + other.z)

    def __lt__(self, other):
        return self.x < other.x

    def __gt__(self, other):
```

```
        return self.x > other.x

    def __eq__(self, other):
        return self.x == other.x

    def quadrant(self):
        if self.x > 0 and self.y > 0:
            print("Quadrant 1")
        elif self.x < 0 and self.y > 0:
            print("Quadrant 2")
        elif self.x < 0 and self.y < 0:
            print("Quadrant 3")
        elif self.x > 0 and self.y < 0:
            print("Quadrant 4")
        else:
            print("On an axis")

    def collinear(self, other):
        if self.x * other.y == self.y * other.x:
            print("Collinear Points")
        else:
            print("Not Collinear")

# Testing the class
p1 = Point(2, 3, 4)
p2 = Point(1, 2, 3)

p1.show()
p1.inc()
```

```
p1.show()
p1.dec()
p1.show()

p3 = p1 + p2
p3.show()

print(p1 < p2)
print(p1 > p2)
print(p1 == p2)

p1.quadrant()
p1.collinear(p2)
```

#Output:

```
Point: (2, 3, 4)
Point: (3, 4, 5)
Point: (2, 3, 4)
Point: (3, 5, 7)
False
True
False
Quadrant 1
Not Collinear
```

Q7. Programs for parsing of data, validations like Password, email, URL, PAN, Mobile number, IP address. Programs for Pattern finding should be covered.

Write regular expression for:

1. To extract year, month and date from a string.
2. To Extract only 3 digit number from string.
3. To Extract all of the words and numbers from string.
4. To Find out all of the words, which start with a vowel.

→(1)

```
import re
date = input("Enter date in YYYY-MM-DD format: ")
pattern = r"(\d{4})-(\d{2})-(\d{2})"
match = re.search(pattern, date)
if match:
    print("Year :", match.group(1))
    print("Month:", match.group(2))
    print("Date :", match.group(3))
else:
    print("Invalid Date Format")
```

#Output:

```
Enter date in YYYY-MM-DD format:  2003-04-26
Year : 2003
Month: 04
Date : 26
```

→(2)

```
import re
text = input("Enter string: ")
pattern = r"\b\d{3}\b"
result = re.findall(pattern, text)
print("3-Digit Numbers:", result)
```

#Output:

```
Enter string:  himanshu 111 hk 888 11
3-Digit Numbers: ['111', '888']
```

→(3)

```
import re
text = input("Enter string: ")
pattern = r"\w+"
```

```
result = re.findall(pattern, text)

print("Words and Numbers:", result)
```

#Output:

```
Enter string:  Himanshu 123 $$##
Words and Numbers: ['Himanshu', '123']
```

→(4)

```
import re

text = input("Enter string: ")

pattern = r"\b[aeiouAEIOU]\w*\b"

result = re.findall(pattern, text)

print("Words starting with vowel:", result)
```

#Output:

```
Enter string:  Himanshu and vikash
Words starting with vowel: ['and']
```

Q8. 1.Write a regular expression to extract to Validate IP Address in Python.

2. Write regular expression to validate email address.

3. Write regular expression to validate URL.

4.Write regular expression to validate PAN Card Number in Python.

→(1)

```
import re

ip = input("Enter IP Address: ")

pattern = r"^((25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)(\.)){3}(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)$"

if re.match(pattern, ip):

    print("Valid IP Address")

else:

    print("Invalid IP Address")
```

#Output:

```
Enter IP Address:  111.222.33.4
Valid IP Address
```

→(2)

```
import re

email = input("Enter Email ID: ")

pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"

if re.match(pattern, email):

   print("Valid Email Address")

else:

   print("Invalid Email Address")
```

#Output:

```
Enter Email ID:  himanshukhatik70@gmail.com
Valid Email Address
```

→(3)

```
import re

url = input("Enter URL: ")

pattern = r"^(https?:\/\/)?(www\.)?[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"

if re.match(pattern, url):

   print("Valid URL")

else:

   print("Invalid URL")
```

#Output:

```
Enter URL:  https://www.google.com
Valid URL
```

→(4)

```
import re
```

```
pan = input("Enter PAN Number: ")
pattern = r"^[A-Z]{5}[0-9]{4}[A-Z]{1}$"
if re.match(pattern, pan):
    print("Valid PAN Number")
else:
    print("Invalid PAN Number")
```

#Output:

```
Enter PAN Number:  ABCDE1234F
Valid PAN Number
```

Q9. An ABC company wants to perform the following tasks:

1. Copy the contents of 'FILE1.txt' to 'FILE2.txt'.

2. Count the number of lines, characters, special symbols for a given file.

Write a python program using multithreading concept to perform above operations

simultaneously. Accept the filenames from user.

(HINT: Thread_obj=Thread(target-Func_A, args=(FILE1, FILE2)) where Thread_obj is thread object, Func_A is function to perform the specific task, FILE1, FILE2 are the file namespassed in the form of tuple as parameter to function.

→

```
import threading
import re

def copy_file(src, dest):
    try:
        with open(src, "r") as f1, open(dest, "w") as f2:
            data = f1.read()
            f2.write(data)
        print(f"File copied from {src} to {dest}")
    except FileNotFoundError:
```

```
        print("Source file not found")

def count_file_details(filename):
    try:
        with open(filename, "r") as f:
            data = f.read()
            lines = data.splitlines()
            line_count = len(lines)
            char_count = len(data.replace("\n",""))
            special_count = len(re.findall(r'[^a-zA-Z0-9\s]', data))

        print(f"\nResults for {filename}:")
        print("Lines         :", line_count)
        print("Characters     :", char_count)
        print("Special Symbols:", special_count)

    except FileNotFoundError:
        print("File not found to count")

# Taking filenames from user
file1 = input("Enter source filename: ")
file2 = input("Enter destination filename: ")

# Threads created
t1 = threading.Thread(target=copy_file, args=(file1, file2))
t2 = threading.Thread(target=count_file_details, args=(file1,))

# Starting threads
t1.start()
```

```
t2.start()

# Waiting for threads completion
t1.join()
t2.join()

print("\nAll Tasks Completed!")
```

#Output:

```
Enter source filename:  file1.txt
Enter destination filename:  file2.txt

Results for file1.txt:
Lines          : 3
Characters     : 22
Special Symbols: 3
File copied from file1.txt to file2.txt

All Tasks Completed!
```

Q10. Write a demo Program for synchronization using RLock. Accept the two numbers from user and calculate factorial of both the numbers.

(Hint: use Rlock(), aquire(), release() methods)

→

```
import threading
lock = threading.RLock()
def factorial(n):
    lock.acquire()
    try:
        fact = 1
        for i in range(1, n + 1):
            fact *= i
        print(f"Factorial of {n} = {fact}")
    finally:
```

```
    lock.release()
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
t1 = threading.Thread(target=factorial, args=(num1,))
t2 = threading.Thread(target=factorial, args=(num2,))
t1.start()
t2.start()
t1.join()
t2.join()
print("Both factorial operations completed successfully!")
```

#Output:

```
Enter first number:  5
Enter second number:  6
Factorial of 5 = 120
Factorial of 6 = 720
Both factorial operations completed successfully!
```

Q11. Write a demo program that reads the contents of the file and counts the occurrences of each letter. Prompt the user to enter the file name.

→

```
def count_letters(filename):
  try:
    with open(filename, "r") as file:
      data = file.read().lower()
      letter_count = {}
      for ch in data:
        if ch.isalpha():
          letter_count[ch] = letter_count.get(ch, 0) + 1
      print("\nLetter Occurrences:")
      for letter, count in sorted(letter_count.items()):
```

```
            print(f"{letter} : {count}")
    except FileNotFoundError:
        print("File not found!")
filename = input("Enter filename: ")
count_letters(filename)
```

#Output:

```
Enter filename:  file1.txt

Letter Occurrences:
a : 2
h : 3
i : 2
k : 2
m : 1
n : 1
s : 1
t : 1
u : 1
```

Q12. Perform the following operations using MongoDB Atlas/Compas/shell

1. Create collection 'emp' and insert five documents.

2. Program for performing CRUD (Create, Read, Update, Delete) operation. Use various methods used for CRUD.

Hint:

1. Insert-insert(), insertOne(), insertMany()

2. Update update(), updateOne(), updatemany()

3. Read - find()

4. Delete deleteOne(), deleteMany()

→

```
client = MongoClient("mongodb://localhost:27017/")
db = client["company"]
col = db["employee"]

def insert_record():
```

```
    name = input("Enter name: ")
    salary = int(input("Enter salary: "))
    col.insert_one({"name": name, "salary": salary})
    print("Record Inserted Successfully")

def display_records():
    print("\n--- Employee Records ---")
    for x in col.find():
        print(x)

def update_record():
    name = input("Enter name to update salary: ")
    new_sal = int(input("Enter new salary: "))
    col.update_one({"name": name}, {"$set": {"salary": new_sal}})
    print("Record Updated Successfully")

def delete_record():
    name = input("Enter name to delete: ")
    col.delete_one({"name": name})
    print("Record Deleted Successfully")

while True:
    print("\n1.Insert\n2.Display\n3.Update\n4.Delete\n5.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1: insert_record()
    elif ch == 2: display_records()
    elif ch == 3: update_record()
    elif ch == 4: delete_record()
```

```
else:
    print("Program Ended")
    break
```

#Output:

```
1.Insert
2.Display
3.Update
4.Delete
5.Exit
Enter choice:  1
Enter name:  Aman giri
Enter salary:  35000
Record Inserted Successfully

1.Insert
2.Display
3.Update
4.Delete
5.Exit
Enter choice:  2

--- Employee Records ---
{'_id': ObjectId('6929ac3561e918ea8c1b09df'), 'name': 'Himanshu khatik', 'salary': 50000}
{'_id': ObjectId('6929ac9161e918ea8c1b09e1'), 'name': 'Himanshu khatik', 'salary': 50000}
{'_id': ObjectId('6929acd261e918ea8c1b09e4'), 'name': 'Aman giri', 'salary': 35000}

1.Insert
2.Display
3.Update
4.Delete
5.Exit
Enter choice:  4
Enter name to delete:  Aman giri
Record Deleted Successfully
```

```
1.Insert
2.Display
3.Update
4.Delete
5.Exit
Enter choice:  2

--- Employee Records ---
{'_id': ObjectId('6929ac3561e918ea8c1b09df'), 'name': 'Himanshu khatik', 'salary': 50000}
{'_id': ObjectId('6929ac9161e918ea8c1b09e1'), 'name': 'Himanshu khatik', 'salary': 50000}

1.Insert
2.Display
3.Update
4.Delete
5.Exit
Enter choice:
```